is too large, LM Studio can use retrieval-augmented generation, or RAG, to select relevant passages and provide those to the model. The document processing remains local.

You do not need to understand how to build RAG yet. Treat the feature as a ready-made tool.

Step-by-step: interrogate a PDF

- **Step 1: Pick the right document.** Start with a clean text-based PDF such as a report, manual or paper. Avoid a badly scanned 300-page book for your first experiment.
- **Step 2: Start a new chat.** Load a capable model with a context setting appropriate to your available memory.
- **Step 3: Attach the PDF.** Wait until the application has processed it before asking questions.
- **Step 4: Begin with extraction, not interpretation.** Ask: "List the five revenue figures given in this document and identify the section in which each appears."
- **Step 5: Move to synthesis.** Try: "What are the three main reasons the report gives for the decline? Use only the attached document."
- **Step 6: Force uncertainty.** Add: "If the document does not contain enough information, say that explicitly."
- **Step 7: Verify against the source.** Open the relevant pages and check the answer.

LM Studio itself advises giving retrieval queries enough specific context – terms, ideas and phrases likely to appear in the document – because this can improve the chance that useful passages are retrieved.

Do not confuse RAG with magical document comprehension. If retrieval chooses the wrong passage, the model may answer from incomplete evidence. If the PDF is an image scan, text extraction may fail. If tables are complex, values may be separated from their headers.

A useful workflow therefore asks the local model to find, organise and explain, while you retain responsibility for verification.

3. Turn it into a coding partner

Coding is particularly attractive locally because source code can be sensitive and because a model does not need internet access to explain a function that is sitting in front of it.

Start with ordinary chat rather than trying to replace your IDE.

Paste a short function and ask:

"Explain what this Python function does. Identify possible logic errors and edge cases. Do not rewrite it yet."

Then ask for a corrected version.

The two-stage process matters. It forces you to inspect the diagnosis before accepting generated code.

Specialist coding models can improve the experience. Qwen3-Coder, for example, is explicitly designed for coding and



local development, with current variants supporting hundreds of programming languages and long contexts. But model size still matters: the best coding model on paper is useless if it runs too slowly on your machine.

Step-by-step: debug locally

- **Step 1:** Paste the smallest reproducible chunk of code.
- **Step 2:** Include the exact error message rather than paraphrasing it.
- **Step 3:** State the language, version and relevant library versions.
- **Step 4:** Ask the model to explain the likely cause before changing code.
- **Step 5:** Request the smallest fix that addresses that cause.
- **Step 6:** Run the code yourself. Never assume generated code is correct because it looks polished.
- **Step 7:** If it still fails, give the model the new error and the exact modified code.

Local models are also useful for generating regular expressions, explaining SQL, drafting shell commands, writing unit-test cases, documenting functions and translating code between languages.

Be especially cautious with commands that delete files, modify permissions, install unknown packages or touch production systems. A local hallucination can damage your machine just as effectively as a cloud hallucination.

4. Give your AI eyes

To work with screenshots, photographs, charts or diagrams, you need a vision-language model rather than a text-only LLM.

Current open models make this increasingly practical. Google's Gemma 4 family accepts text and images across the family and supports more than 140 languages; llama.cpp's current multimodal layer supports image, audio and video input for compatible models.

LM Studio also supports image input with compatible VLMs and accepts JPEG, PNG and WebP through its local model interfaces.